Task 2 – QA Strategy & Thinking

---

1. Where do I start?

I wouldn't start by writing automation scripts immediately. First, I need to understand what we are releasing and where the biggest risks are.

Understand the product and business

My first step would be to sit with the product manager, developers, designers and whoever owns the trading or financial operations side of the business.

I'd want answers to a few questions:

- What types of trading are supported?
- How does a user deposit and withdraw funds?
- How are orders created, processed, executed and cancelled?
- What happens when an order is partially filled?
- How are failed transactions handled?
- What happens if the network disconnects while a transaction is in progress?
- Are there any regulatory or compliance requirements that must be met before launch?

I would also ask for access to the staging environment, API documentation, architecture diagrams, existing tickets and any available design specifications.

Identify critical user journeys

I would map the main customer journeys, starting from account creation through to trading and withdrawing funds.

For example:

1. User registration and login.
2. Identity verification and account approval.
3. Deposit funds.
4. View wallet balance.
5. View live market prices.
6. Place a buy or sell order.
7. Check order execution and transaction history.
8. Cancel an open order.

9. Withdraw funds.

I would pay particular attention to the points where money moves or balances change.

Understand the current quality of the product

I would spend some time exploring the application on both iOS and Android.

I wouldn't just follow the happy path. I'd deliberately try things such as exploratory testing.

- Tapping the buy button multiple times.
- Disconnecting the network during an order.
- Closing and reopening the application during a transaction.
- Trying to trade with insufficient funds.
- Entering invalid order quantities.
- Checking whether wallet balances and transaction history remain consistent.

I would also review the recent development activity to understand which areas have changed frequently and where the team has known issues. (will leverage previous incident reports or get this data based on the impact areas assessment done by development teams)

At the end of this initial assessment, I want three things:

- A list of critical business journeys.
- A risk-based test plan.
- A clear understanding of what must be fixed before public release.

---

2. How would I approach testing this app?

I would use a risk-based testing approach.

Not every feature carries the same level of risk. A minor formatting issue on a profile screen is different from a bug that allows a user to place an incorrect order or lose funds.

I'd divide the testing into the following areas.

A. Functional testing

This would cover the main business functionality:

- Registration, login and logout.
- KYC and account verification.
- Deposits and withdrawals.

- Wallet balance and transaction history.
- Market data and price updates.
- Buy and sell orders.
- Order cancellation.
- Order status and execution.
- Error handling and validation.

For trading functionality, I would cover both positive and negative scenarios.

For example, when placing an order, I would verify:

- The correct trading pair is selected.
- The order quantity and price are correct.
- The available balance is validated.
- The order is submitted only once.
- The order status is updated correctly.
- The wallet balance is adjusted correctly after execution.
- Transaction history reflects the same transaction.
- Multi currency cases.

I would also test partial fills, rejected orders, cancelled orders and insufficient balance scenarios.

B. Financial integrity testing

This is probably the most important area for this application.

I would work with developers and the backend team to understand how balances and transactions are maintained.

Some scenarios I would specifically test:

- The same order should not be processed twice.
- Retrying a failed request should not result in duplicate transactions.
- Wallet balances should not become negative unless the product explicitly supports an appropriate borrowing or margin model.
- The available balance and actual balance should be consistent with the transaction state.
- A failed transaction should not incorrectly deduct funds.
- A successful withdrawal should be reflected consistently across the wallet and transaction history.
- Decimal precision and rounding should be handled correctly.
- Concurrent requests should not cause incorrect balance calculations.

I would also ask the team how they handle transaction idempotency, reconciliation and audit trails.

For financial transactions, I would not rely only on what the mobile application displays. Wherever possible, I would validate the backend response and persist transaction state as well.

C. API testing

I would introduce API testing early because a mobile application depends heavily on backend services.

I would cover:

- Authentication and authorization.
- Request and response validation.
- Mandatory field validation.
- Invalid input handling.
- HTTP status codes and error responses.
- Transaction creation and status updates.
- Duplicate request handling.
- Rate limiting.
- Session expiration.

For example, if the mobile application shows a successful order, I would verify that the backend has actually accepted the order and that the response contains the expected order identifier and status.

D. Mobile-specific testing

Since we are releasing on iOS and Android, I would not assume that passing on one platform means the other platform is good.

I would test on a representative device matrix covering:

- iOS and Android.
- Supported OS versions.
- Different screen sizes.
- Different network conditions.
- Background and foreground transitions.
- Application restart and session recovery.
- Push notifications, if applicable.
- Permission handling.
- Application upgrade scenarios.

I would also test interruptions such as incoming calls, device locking and network switching during critical transactions.

E. Security and non-functional testing

For a fintech application, security cannot be treated as an optional final activity.

I would involve the security team or an external specialist where necessary and focus on:

- Secure authentication and session management.
- Biometric authentication, if supported.
- Authorization and access control.
- Secure storage of tokens and sensitive information.
- Sensitive data exposure in logs.
- Certificate and transport security.
- Protection against repeated transaction submissions.
- Basic performance and load testing of critical APIs.

I would also check how the application behaves during market volatility or unusually high order volumes.

---

3. What does QA look like inside a sprint?

I believe QA should be involved from the beginning of a story, not only when development is complete. I would support the shift left testing approach.

This is how I would integrate QA into the sprint.

Stage 1: Ticket creation and refinement

When a new story is discussed, QA participates in refinement.

I would look at:

- Whether the requirement is clear.
- Whether acceptance criteria cover negative scenarios.
- What could go wrong.
- Whether there are API or database dependencies.
- Whether the feature impacts existing functionality.
- What test data or environment access is required.

For example, if the story is about placing a buy order, I would ask what should happen when the user has insufficient funds, when the price changes before execution, or when the request times out.

I would also identify the testability requirements early.

Stage 2: Test design

Once the requirement is understood, I would prepare test scenarios and identify which ones should be automated.

I generally classify scenarios into:

- Positive scenarios.
- Negative scenarios.
- Boundary conditions.
- Integration scenarios.
- Security and financial integrity scenarios.

I would also identify the areas requiring exploratory testing.

Stage 3: During development

I would stay connected with developers rather than waiting for a completed build.

I would encourage:

- Unit tests for business logic.
- API contract validation.
- Code reviews that consider testability.
- Early deployment to QA environments.
- Developer-level testing before handing over a feature.

If something is unclear, I would resolve it during development rather than discovering the ambiguity during final testing.

Stage 4: Feature testing

Once a build is available, I would validate the acceptance criteria first.

Then I would cover negative cases, integration points and relevant exploratory scenarios.

Defects would be logged with:

- Clear reproduction steps.
- Expected and actual results.
- Screenshots, videos or logs.
- Device and OS details.
- Severity and business impact.

For financial issues, I would immediately communicate with the developer and product owner instead of waiting for the next defect triage meeting.

Stage 5: Fix verification and regression

Once a defect is fixed, I would retest the original scenario and check related functionality.

For example, a fix to wallet balance calculation should trigger regression around deposits, withdrawals, order placement, order cancellation and transaction history.

At the end of the sprint, I would share a quality summary covering:

- Stories tested.
- Test execution status.
- Defects raised and resolved.
- Outstanding risks.
- Automation coverage.
- Any areas that need additional attention.

My expectation is that QA provides visibility into product quality throughout the sprint, not just a pass/fail result at the end.

---

4. What would my ideal regression suite look like?

I would build the regression suite in layers rather than trying to automate everything.

The objective is to get fast and reliable feedback while protecting the critical business functionality.

Layer 1: Smoke tests

These should be quick and run against every new build.

Examples:

- Application launches successfully.
- User can log in.
- Dashboard loads.
- Wallet balance is visible.
- Market data is available.
- Trading screen opens.
- User can access order history.

If a critical smoke test fails, there is little value in proceeding with the full regression suite.

Layer 2: Critical business regression

This is the most important suite.

It should cover:

- Deposit and withdrawal flows.
- Wallet balance calculations.
- Buy and sell orders.
- Order execution.
- Partial fills.
- Order cancellation.
- Insufficient balance.
- Duplicate order prevention.
- Transaction history consistency.

I would automate these scenarios as a priority, with suitable test accounts and controlled test funds.

Layer 3: Functional regression

This would cover the remaining application functionality:

- Registration and KYC.
- Profile management.
- Notifications.
- Settings.
- Search and market filters.
- Session management.
- Error messages and validations.

Layer 4: Non-functional regression

This would include:

- API performance checks.
- Application launch time.
- Network interruption handling.
- Basic security checks.
- Memory and crash monitoring.
- Device compatibility.

Automation strategy

I would use a combination of tools based on the application architecture.

Area	Possible tool
iOS and Android UI automation	Appium

API automation	Playwright API / REST Assured
Performance testing	k6/Locust
Mobile crash monitoring	Firebase Crashlytics or equivalent
CI/CD	GitHub Actions / Jenkins

I would not automate every scenario through the UI. For example, balance calculations and transaction rules are better validated through API and service-level tests wherever possible.

I would also avoid tests that depend on live market prices or uncontrolled real-money transactions. Those should use a controlled test environment with deterministic data.

Execution frequency

- Smoke suite: Every build.
- Critical API and financial regression: Every build or deployment to QA.
- Full functional regression: Before release.
- Device compatibility: On planned builds and release candidates.
- Performance and security testing: At appropriate milestones and before release.

The suite should be reliable. A flaky regression suite that the team stops trusting is not useful, regardless of how many tests it contains.

---

5. What would keep me up at night about this app?

Honestly, it would be the possibility of a customer losing money because of a defect that we failed to identify.

There are a few scenarios I would be particularly concerned about.

1. Duplicate transactions

A user taps Buy, the request times out, and they tap again. If the backend processes both requests, the user may end up with two orders.

I would like to verify idempotency and duplicate request handling.

2. Incorrect wallet balances

The application shows one balance, while the backend maintains another.

This could happen due to incorrect calculations, concurrent transactions, rounding errors or delayed updates.

I would want strong balance consistency checks.

3. Incorrect order execution

A user places an order at a particular price, but the application displays an incorrect status or the backend executes it incorrectly.

I would specifically test order lifecycle transitions, partial fills, rejected orders and cancellation.

4. Network failures during transactions

Mobile networks are unpredictable.

A user may lose connectivity immediately after placing an order. The application must not simply assume that the transaction failed or encourage the user to submit it again.

The system should be able to recover the actual transaction status from the backend.

5. Security vulnerabilities

Unauthorized users accessing accounts, session hijacking, exposed tokens or weaknesses in withdrawal authorization could have serious consequences.

I would treat these as release-blocking risks.

6. Production readiness

I would also want to know what happens when something goes wrong after release.

- Is transaction monitoring available?
- Can the business identify failed or duplicate transactions?
- Is there an incident response process?
- Can a problematic feature be disabled?
- Is there a rollback or release-disable mechanism?
- Is customer support prepared to handle transaction-related complaints?

For me, release readiness is not just about whether the application passes testing. It is also about whether the business is prepared to handle failures.

Deliverable 1 – Test Plan

Objective

Validate that the mobile trading application is functionally correct, secure, stable and reliable enough for its first public release.

Test area	Scope	Priority
Authentication	Login, logout, session handling	High
KYC	Verification and approval flows	High
Wallet	Balance, deposits and withdrawals	Critical
Trading	Buy, sell, cancel and execution	Critical
Order management	Partial fills, rejected and pending orders	Critical
API	Contracts, validation and error handling	High
Security	Authentication, authorization, data protection	Critical
Mobile compatibility	iOS, Android and device coverage	High
Performance	API response time and application stability	High
Notifications	Order and transaction notifications	Medium

Test approach

- Functional testing on iOS and Android.
- API and integration testing.
- Exploratory testing around financial transactions.
- Negative and boundary testing.
- Security testing with appropriate specialist involvement.
- Performance testing for critical APIs.
- Regression testing on release candidates.

Test environments

I would prefer a dedicated staging environment with:

- Test user accounts.
- Controlled wallet balances.

- Simulated or sandbox deposits and withdrawals.
- Predictable market data.
- Ability to create, execute and cancel test orders.
- Access to application and backend logs.

No uncontrolled real-money transactions should be used for routine testing.

Deliverable 2 – Release Readiness Checklist

I would use this checklist during the release review.

Functional readiness

- Registration and login work correctly.
- KYC flow is validated.
- Deposit and withdrawal flows are tested.
- Wallet balances are consistent.
- Buy and sell orders work correctly.
- Order cancellation is validated.
- Partial fills and rejected orders are tested.
- Transaction history reflects the correct status.

Financial integrity

- Duplicate transaction prevention is verified.
- Balance calculations are validated.
- Decimal precision and rounding are verified.
- Failed transactions do not incorrectly deduct funds.
- Concurrent transaction handling is tested.
- Transaction audit trails are available.
- Reconciliation process is understood and validated.

Mobile readiness

- iOS supported versions tested.
- Android supported versions tested.
- Different screen sizes validated.

- Network interruption scenarios tested.
- Background and foreground transitions tested.
- No critical crashes observed.

Security and performance

- Authentication and authorization checks completed.
- Sensitive data is not exposed in logs.
- Session handling is validated.
- Critical API performance is acceptable.
- Basic security assessment completed.
- No unresolved critical security issues.

Operational readiness

- Monitoring and alerting configured.
- Crash reporting enabled.
- Incident response ownership identified.
- Customer support is prepared.
- Rollback or release-disable plan available.
- Known issues documented.
- Product and engineering sign-off received.

Final QA recommendation

I would not recommend public release with unresolved critical or high-severity defects affecting financial transactions, account security or data integrity.

Any accepted residual risk should be documented, reviewed by the appropriate business and engineering owners, and supported by a mitigation plan.

QA should provide the release recommendation, while the final business decision should be made by the accountable release stakeholders.

Deliverable 3 – Risk Matrix

I would prioritize risks based on their likelihood and business impact.

Risk	Likelihood	Impact	Priority	Mitigation
Duplicate order execution	Medium	Critical	Critical	Idempotency and duplicate request testing
Incorrect wallet balance	Medium	Critical	Critical	Balance reconciliation and concurrent transaction tests
Unauthorized withdrawal	Low/Medium	Critical	Critical	Security and authorization testing
Incorrect order execution	Medium	Critical	Critical	Order lifecycle and API validation
Network failure during order placement	High	High	Critical	Retry and transaction recovery testing
Application crash	Medium	High	High	Device testing and crash monitoring
Incorrect market data	Medium	High	High	Validate price source, freshness and consistency
Notification delay	Medium	Medium	Medium	Notification and event delivery testing
UI layout issue	Medium	Low	Low	Device and screen-size validation

Note: These are initial qualitative estimates. I would refine likelihood and impact after reviewing architecture, transaction flows, existing defects and production controls.

Framework and Design Decisions

Since this is a mobile trading application, I would avoid choosing a framework before understanding the application technology and the team's existing setup.

My initial choices would be:

- **Appium:** Cross-platform UI automation for native iOS and Android applications.
- **Playwright API or REST Assured:** Backend API validation, depending on the team's preferred language and existing infrastructure.

- **k6:** Performance testing of critical APIs.
- **GitHub Actions or Jenkins:** Automated execution through CI/CD.
- **Allure or Playwright HTML reports:** Test execution reporting and failure evidence.

I would use a layered automation design:

1. Test data and environment configuration.
2. API clients and service-level helpers.
3. Page/screen objects for mobile UI interactions.
4. Reusable business workflows.
5. Test specifications.
6. Reporting and execution configuration.

The aim is to keep business logic separate from UI interactions and make tests easy to maintain.

Assumptions

Since detailed product documentation was not provided, I have made the following assumptions:

1. The application supports user registration, KYC, deposits, withdrawals and trading.
2. The application communicates with backend APIs for account, wallet and order management.
3. A staging environment is available or can be created.
4. The company can provide test accounts and controlled funds.
5. The product has a defined set of supported iOS and Android versions.
6. The company has access to the relevant security, compliance and operational stakeholders.

If any of these assumptions are incorrect, I would revisit the test scope and release risks accordingly.

Final Thoughts

If I had only two weeks, I would spend the first couple of days understanding the product and identifying the critical risks. I would then focus on testing the financial workflows, API integrations and mobile-specific failure scenarios.

In parallel, I would establish basic smoke automation, a repeatable regression process and a release readiness checklist.

I wouldn't measure success by the number of test cases written or automated. I would measure it by whether we have identified the important risks, whether the team has confidence in the critical workflows, and whether the business has enough information to make a responsible release decision.

For an application dealing with people's money, I would much rather raise a release concern with evidence than give a premature green signal just to meet a deadline.